Toward fewer than 196 states: comparison, source reductions, and remaining proof obligations

Research prepared for Vladimir Reshetnikov

13 November 2026

Abstract

We compare the designated incoming 208-state machines, James Chang's 209-state predecessor, and the published 196-state machine at fixed source revisions. The requested endpoint—an explicit machine with fewer than 196 working states and a PA-provable nonhalting/consistency equivalence—is **not established by this work**. There are nevertheless new, exact source-level results: a 157-node representation of the 196-state machine's register program, down from 159 distinct internal nodes, with the complete 3,812-instruction control graph unchanged; and a simpler 158-node factorization that exposes why an apparently promising 195-state allocation loses an existing decoder helper. We give reproducible comparisons, stronger bounded state-fold screens, a native calculus of HA strength obtained from a ConciseIPRA idea, and a canonical proof-stack construction for the missing internal soundness direction. Source nodes, physical states, bounded experiments, written mathematical arguments, and kernel-checked theorems are kept distinct throughout.

1 Outcome and exact scope

The requested machine is a deterministic, single-tape, two-symbol Turing machine, starting on a two-way infinite all-zero tape. Working states exclude a separate halting control. A successful construction would give an exact table T, an independently checkable count $|Q_T| < 196$, and

$$\mathsf{PA} \vdash \neg\,\mathrm{Halt}(T) \leftrightarrow \mathrm{Con}(\mathsf{PA}).$$

No table in this packet is represented as meeting that endpoint. The unmodified upstream tables are included as comparison inputs. The new 157- and 158-node objects are register-program source trees, not Turing machines with those numbers of states.

The principal positive result is stronger than an agreement on sampled executions. All operations and all ordinal successor edges of the register program agree exactly. Hence its transition function and every execution, for every natural-valued register vector, agree. What is missing is a physical allocation that realizes the source saving while preserving the binary tape interpreter, startup, and arithmetic theorem.

The theorem inventory for the published PA196 also requires care. It lists

$$\mathsf{PA} \vdash N_{196} \to \mathrm{Con}(\mathsf{PA}), \qquad N_{196} := \neg H_{196},$$

where H_{196} is the literal arithmetic halting sentence. It lists external nonhalting and two-sided PA independence as well. It does not list the internal converse $\mathrm{Con}(\mathsf{PA}) \to N_{196}$ [2, 3]. This is a

distinction in the supplied proof interface, not an assertion that the converse is false or unprovable. ProveIt's existing bridge audit already records this distinction [7]; it is not claimed as a new discovery here.

1.1 Pinned inputs

The two current branches were resolved before comparison:

James Chang, PA196
`407cdc356f6b586635e71ac92be394255f4bfea4`.

James Chang, requested PA209 predecessor
`69d98241a721ead20b4f0b17df9062950b40fbfe`.

ProveIt, incoming designs and ConciseIPRA
`f1f076e31995f10460838d1846456f4bb9c8ae80`.

The PA196 literal table has SHA-256

`e8bbccd4cfa325251ca95066db116b5aa766e404b0971c33c02772be1c3617a8`.

Its source `PA196/Program.lean` has SHA-256

`8c7c6c6884ecbb8f56e893404bb81162db15ef3dd6fd3212881a93994d63d872`.

The source/table publication checks and generated-source checks were rerun successfully. The full Lean project was not rebuilt in this environment; no new Lean kernel verification is claimed.

2 What the existing designs actually share

Design	Working states	Rows	Source sites	Max. source path
Incoming U208, class A	208	416	3,909	42
Incoming U208, class B	208	416	3,909	41
Chang PA209	209	418	3,948	—
Chang PA196	196	392	3,812	38

Table 1: Physical-state counts for existing inputs. A missing entry is not used in any conclusion. Source sites are instruction occurrences, not distinct source DAG nodes.

2.1 Nine designated U208 finals reduce to two table classes

The comparison canonicalizes a rooted transition table by visiting successor states in a fixed read-symbol order, starting at its declared initial state. It retains writes, directions, and the distinguished halt status. Full canonical transition encodings are compared; hashes are only compact identifiers, not substitutes for those comparisons. Under this exact notion of state renaming, the nine designated final inputs split into two classes, of sizes five and four. The archive member names, original hashes, complete renaming maps, and canonical rows are in `comparison/comparison.json`.

The two source representatives have the same 3,909 operations in the same ordinal order and the same 5,083 nonhalting successor edges. Exactly 720 leaf paths differ. Thus their physical difference is a layout difference, not a different arithmetic program. Both have maximum path-zero run six.

This rechecks the corresponding earlier comparison independently; the two-class observation itself is not new.

The U208 final reduction merges native state 211 into 172 in its particular U209 parent. The relevant mixed-entry pattern includes

$$(163, 0, 0, L, 211), \qquad (163, 1, 1, R, 211).$$

Its two histories permit a different zero row for the keeper, with an existing helper distinguishing the cases. The designated U208 finals do not supply nine independent savings to add together. Nor is this U209 parent identified merely by its number with Chang's PA209.

2.2 PA209 to PA196 changes both logic and layout

Chang's paper describes three changes in the axiom-emitting program [1]. For Cantor pairing

$$p(a, b) = \frac{(a + b)(a + b + 1)}{2} + b,$$

the implication, membership, and equality constructors remain

$$I(a, b) = 4p(a, b), \quad M(a, b) = 4p(a, b) + 1, \quad E(a, b) = 4p(a, b) + 3.$$

The universal constructor changes to

$$U(a, B) = 4p(B, a) + 2.$$

The body and label are transposed together in the encoder and decoder. The program removes the old seventh head, adds guarded equality vacuity to the gate, changes head order, and exchanges selected antecedents. The old uses of the removed head are recovered by derivations, including reverse membership rewriting.

The resulting axiom body has 201 macro tokens and 68 internal binary grammar nodes. Six pairs share states, giving 62 body-reader states. In the complete machine the accounting is

$$36 \text{ engine} + 13 \text{ leaf routing} + 147 \text{ source readers} = 196.$$

The complete source DAG has 159 distinct internal nodes; contextual sharing is why this is not the number of source-reader states.

The exact U208 mixed-entry motif above does not occur in PA196, even up to state renaming: no state has a zero row writing zero and moving left and a one row writing one and moving right to the same working target. The existing comparison and accompanying scan establish precisely that literal obstruction [6]. It does not exclude other contextual folds.

3 A new 157-node source representation

3.1 Exact witness and verification

Long sequences of admissible source reassociations produce `source_reductions/source157.json`. It is a complete ordered list of leaf paths and instructions. Its SHA-256 is

`615871ff18b65818b06465ecae981fced6c2a78a82f8b6519fe9033b6bfcfa44`.

Source tree	Internal nodes	Sites	Max. depth	Max. zero run
Published	159	3,812	38	6
New compact witness	157	3,812	37	6
Layout-estimate witness	158	3,812	37	6
Minimal factorization	158	3,812	37	8

Table 2: Source-tree measurements only. None is a physical-state count.

Independent parsing and ordinal-interval resolution give:

Every source has 1,516 increments, 1,147 conditional decrements, 1,069 backward jumps, 79 forward jumps, and one halt instruction. For each source, every instruction's register operand and every zero/nonzero or unconditional successor agrees with the published ordinal graph. The compact 157-node witness preserves the source seed at site 25, path `100000`. The separate layout-estimate witness places site 25 at `1000000`; a trailing-zero address change alone neither proves nor disproves physical startup equivalence.

The independent audit also runs the source from registers $(2, 0, 0, 0, 0, 0, 0, 0, 0)$ at site 25. After 100,000 steps every graph gives site 693 and register vector

$$(4, 364, 0, 0, 0, 0, 1266, 6178, 0).$$

This run is a control. The all-time source equivalence follows from exact graph equality, not from this finite execution.

Proposition 3.1 (Source execution equivalence)**.** *Let R be the published PA196 register program and R' any of the three new source witnesses. At corresponding ordinal program counters, their one-step transition functions on $\mathbb{N}^9$ are equal. Consequently their executions agree for every initial register vector and every finite number of source steps, including their halt status.*

Proof. For an increment the changed register and successor are identical. For a decrement the tested register and both successors are identical. The jump and halt cases are identical as well. These are all instructions. Induction on the number of steps proves the second statement. The finite graph check and this induction are arithmetizable in PA; this observation does not claim that a PA proof certificate has been exported here. □

3.2 Why the reassociation rule is valid

An internal tree node represents sequencing together with structured escape levels. Write $f(T), b(T)$ for escaping forward and backward depth. An increment and halt have depths $(0, 0)$; a decrement has $(1, 0)$; a forward jump of depth k has $(k, 0)$; a backward jump has $(0, k)$. For a pair,

$$f((A, B)) = \max(f(A) - 1, 0, f(B)),$$
$$b((A, B)) = \max(b(A), b(B) - 1, 0).$$

Lemma 3.2 (Directional reassociation)**.** *The rewrite $((A, B), C) \leftrightarrow (A, (B, C))$ preserves the ordinal instruction graph when $f(A) = 0$ and $b(C) = 0$. The middle factor need not be closed.*

Proof. The ordered instruction word is unchanged. Paths within a factor retain their internal ancestors. A forward escape from the first factor is the only forward case whose counting of outer left edges could change its target; the first condition excludes such an escape. A backward escape

from the last factor is the analogous backward case and is excluded by the second condition. Escapes from the middle factor cross the corresponding factor boundaries in the same ordinal order. A direct case split on forward and backward escape levels gives the same predecessor/successor interval endpoints. The supplied replay additionally checks every edge of each concrete witness, so the reported finite result does not depend on trusting an informal optimizer transformation. □

Equal finite subtrees cannot be proper subtrees of one another. Applying a rewrite simultaneously to every occurrence of the same subtree therefore rewrites disjoint occurrences. Global rewrites inherit the local lemma. This explains why sequences longer than the earlier restricted three-rotation screen can be productive. No claim of minimum grammar size is made for 157.

4 A simple 158-node factorization, and why it is not PA195

The stochastic witness can be simplified to a readable common-factor idea. Use the upstream node names and put

$$A = \texttt{node32}, \quad B = \texttt{node53}, \quad D = \texttt{node55}, \quad C = \texttt{node65}.$$

Here D pushes and copies the appropriate source register into the accumulator, and C pops and constructs the implication-pair code. All four factors are closed source blocks.

The first reassociation is

$$\underbrace{((A,(B,D)),C)}_{\texttt{node75}} \quad \longmapsto \quad ((A,B),(D,C)).$$

The second exposes the same (D,C) in another part of the axiom body:

$$\underbrace{(D,(((C,X),Y),Z))}_{\texttt{x37}} \quad \longmapsto \quad ((D,C),((X,Y),Z)),$$

where $X = \texttt{x27}$, $Y = \texttt{x19}$, and $Z = \texttt{x35}$. The construction applies the replacements consistently to repeated subtrees. The two occurrences of (D,C) now share one internal source node. Exact reconstruction gives 158 nodes, with unchanged first 50 source paths and unchanged complete ordinal graph.

4.1 The hidden cost of the old helper

The physical machine uses native state 213 for more than reading `node75`. It is also the history helper for a shared native state 69. These are sparse native labels, rather than the dense labels 0–195 in `data/pa196.tm`. The relevant native rows are

State	Read	Write	Move	Target
69	0	0	L	213
213	0	0	R	92
213	1	1	R	90
92	0	0	R	104
90	0	0	R	105

At a current zero, state 69 moves left to reread the preceding tag. A zero tag takes the three-step detour to 104; a one tag takes it to 105. The tape is restored and the head advances past the original current cell.

The original source node 75 happens to implement exactly that two-stage test. Reassociating it changes the required right child of the decoder; the new factor (D, C) does not have the old zero-row behavior of state 90. Thus subtracting one source node while retaining all old sharing savings counts the benefit but omits the cost of replacing a helper.

The restricted physical allocator gives no allocation for this candidate with its fixed compatible-reader, protected-helper, and label-order constraints. With the additional requirement of retaining every original startup row it already has an empty domain. These outcomes are about that allocator, not all possible implementations. The minimal factorization also has an eight-zero source path, so it needs fresh return-scan analysis; the compact 157-node witness has no such increase.

A targeted reassociation search considered all 16,796 binary associations of an expanded eleven-atom prefix, with the old or new node-75 arrangement outside it: 33,592 whole-program cases in total. Of these cases, 17,685 whole-program variants satisfied the demanded helper-pattern filter. Their best state-count estimate was 196. This is a limited negative result, not a theorem that the factorization can never save a state.

5 Broader finite searches and their limits

5.1 Register abstractions

The source was also analyzed from its actual seed using joint finite register abstractions. All nine registers are tracked together. The two-category control distinguishes zero from positive; three categories distinguish $0, 1, \geq 2$; four distinguish $0, 1, 2, \geq 3$.

Categories	Reachable pairs	Checked edges	Forced-zero sites
$0, \geq 1$	274,880	326,688	3773
$0, 1, \geq 2$	4,455,507	5,000,898	7, 3773
$0, 1, 2, \geq 3$	32,947,398	35,969,718	7, 3773

Table 3: Pairs consist of a source site and an abstract nine-register vector. Closure edges include all abstract successors, not sampled traces.

The finer two screens identify only the already known unreachable sites 8, 3774, and 3775. Adding the sign of $r_i - r_j$ to the joint three-category domain was tested for all 36 register pairs, retaining an unbounded equality/order relation for the selected pair. This gave no new forced branch. Abstract halt remains reachable, which is compatible with sound overapproximation: these abstractions cannot prove actual nonhalting. They also cannot exclude a useful invariant in a richer domain.

5.2 Source-conditioned local tape contexts

The new local fold search refines the earlier tape-window abstraction by collecting windows from symbolic, arbitrary-register instruction contracts, the exact startup trace, and conservative generic-loop contexts. The source-derived set contains 2,230 five-cell contexts. Intersecting it with the earlier radius-six blank closure gives 2,219 admitted contexts, compared with 2,774 in that earlier closure. All 4,959 instruction/branch contracts are replayed during extraction, covering 1,185,472 symbolic steps. Independently, 353 concrete complete-frame fixtures totaling 240,902 steps agree with the extracted contexts. The concrete fixtures are controls, not the justification for arbitrary-value coverage.

The local matching template allows a positive block of one to sixteen replacement transitions inside a five-cell window. Its stuttering endpoint must agree exactly in projected control state, head position, and all five cells. It does not allow arbitrary head offsets, long-range tape encodings, or unbounded local detours. In this template, single-helper and two-retained-row screens have no survivor. The larger two-arbitrary-row synthesis examines all 38,220 ordered state folds. Its counters are

Eligible first changed keys	151,154
First-row assignments examined	118,504,736
Eligible second-key cases	732,127
Second-row assignments examined	573,987,568
Representative checks passed	633
Full-context survivors	0

A separate four-state control recovers both a one-helper fold and folds using two changed row assignments. The counters are numbers of tested branches or assignments in the algorithm, not a count of distinct nonisomorphic machines.

These results are bounded search evidence. The symbolic context extraction is executable reasoning with explicit loop abstractions, not a newly kernel-checked invariant theorem. Even if every invariant lemma were formalized, the conclusion would still concern only this particular five-cell, sixteen-step, two-row template. It is not a lower bound of 196 for PA-consistency machines.

6 A concrete use of ConciseIPRA: change the target to HA strength

The supplied iPRA work is relevant, but directly substituting an iPRA proof checker would search for a contradiction in the wrong theory. A useful transfer is to enlarge its induction mechanism while retaining its compact native logical rules. This section gives a written metatheorem for a report-level calculus with *full formula-template equality transport*. It does not claim the corresponding modification of the later `trOne` Lean kernel or an optimized binary TM compiler.

6.1 Syntax and rules

Use numerical terms

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h),$$

where only u is bound in the iterator body h. Native formulas are equations and the constructors $Q_y B$ and $D_{x,y} B$, with distinct bound coordinates x, y. Their ordinary readings are

$$Q_y B := B(0) \to \forall y\, B(y), \qquad D_{x,y} B := \exists x \big(B(x, 0) \to \forall y\, B(x, y)\big).$$

Falsity is $0 = S(0)$. Contexts are finite. Besides hypothesis and computation,

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u],$$

use the following rules, with capture-avoiding substitution.

Transport From $s = t$ and $F(s)$ infer $F(t)$ for any native formula template F.

QE From $Q_y B$ and $B(0)$ infer $B(t)$.

DE From $D_{x,y}B$ and a derivation of A under the extra hypothesis $Q_yB(b,y)$ infer A. The variable b is fresh in the outside context, major premise, and conclusion.

DI From $\Gamma, B(t,\eta) \vdash B(t,Sz)$ infer $\Gamma \vdash D_{x,y}B$, where z is distinct from x,y and is absent from the free variables of Γ, t, and the free parameters of B other than its displayed coordinates x,y, and

$$\eta = \begin{cases} z, & t \text{ is literally } 0, \\ 0, & \text{otherwise.} \end{cases}$$

Call the new calculus W_H. In the corresponding report-level iPRA calculus, progress mode additionally requires the matrix to be a literal equation. The new rule removes that matrix test, and nothing else.

6.2 Recovering the removed zero-witness case rule

The modification changes some old cases, so mere inclusion of rule sets is not enough. Let $e = I(0,0;u.0)$. Computation proves $e = 0$, while e is not syntactically the literal zero. Reflexivity and symmetry follow from computation and full transport without DI.

Suppose an old zero-witness case premise is available:

$$\Gamma, B(0,0) \vdash B(0,Sz).$$

Use nonliteral witness e with the matrix $B(0,y)$ and an unused witness coordinate. This selects case mode and gives $D_{w,y}B(0,y)$. Open it by DE to obtain the hypothesis $Q_yB(0,y)$. Now introduce the original matrix at witness e. Its temporary hypothesis $B(e,0)$ transports to $B(0,0)$; QE yields $B(0,Sz)$; transport back yields $B(e,Sz)$. DI gives $D_{x,y}B$, and DE closes the opening. This works for compound matrices too. Thus every old rule has a uniform replacement, and the inherited native logical macros remain available [8].

6.3 Full induction and the consistency target

For any native formula $A(y)$ and fresh z, a progress premise $\Gamma, A(z) \vdash A(Sz)$ now gives $D_{x,y}A(y)$ at literal-zero witness, where x is unused. DE gives $Q_yA(y)$. With $\Gamma \vdash A(0)$, QE gives $A(t)$ for arbitrary t. Hence full induction is recovered.

Conversely, interpret the native constructors by their displayed readings in HA, and every finite iterator by its primitive-recursive definition. QE, DE, transport, and computation are HA-valid. In DI's progress case use full HA induction. In the case mode, numerical zero/successor cases suffice to derive the displayed closure from its base. Therefore W_H has HA strength under these interpretations, using the inherited logical macro calibration. Standard syntactic negative translation takes a PA refutation to an HA refutation; the other direction uses the classical extension. Composed with the native proof translations, this gives the correct PA consistency target for a future W_H refutation search.

The reference implementation applies the selector change only in DI. It does not perform a textual replacement in the upstream compressed checker, whose fused dispatch expression has additional roles. Nine checks pass, including a compound-progress certificate accepted in HA mode and rejected as the same packet in the iPRA control. This is not a claim that the certificate's conclusion separates the theories. The checker is a readable reference implementation; its rule correctness and equivalence are not newly kernel verified.

Checking a finite certificate is primitive recursive under standard finite-tree encodings: proof traversal, substitution, fresh-name choice, and alpha comparison are structural and do not evaluate

arbitrary iterator values. A binary TM implementation must still include certificate enumeration, parsing, substitution, and its own startup. Python size gives no state bound. No sub-196 compilation of W_H is supplied.

7 The missing PA-internal soundness direction

To complement the upstream theorem, it suffices to prove

$$\mathsf{PA} \vdash H_{196} \to \mathrm{Prov}_{\mathsf{PA}}(\ulcorner \bot \urcorner).$$

The direct approach is syntactic proof production, not a global truth predicate. A literal halting trace must yield a source halt, a completed command run, and a PA refutation. Each transformation must work for arbitrary PA-internal finite codes, including nonstandard codes in nonstandard models.

7.1 A canonical stack invariant

The numerical stack has the alias $p(0,0) = 0$. It is therefore incorrect to assume a proof stack gains one hidden element on every numerical push. Write $\mathrm{hd}(w), \mathrm{tl}(w)$ for Cantor decoding and define

$$L(w) = \min\{k \le w : \mathrm{tl}^k(w) = 0\}.$$

Lemma 7.1 (Canonical proof stack)**.** *The function L is primitive recursive. PA proves the totality and defining properties of its represented graph, and the push/pop contracts for a certificate sequence indexed by the canonical decoded entries, including the exceptional value zero.*

Proof. For $w = p(a,b) > 0$, $b < w$. If $b = 0$ this is immediate; if $b > 0$, then $p(a,b) \ge 2b > b$. Repeated positive tails therefore strictly decrease, so zero is reached within w steps. Bounded iteration and minimization define L. Let the certificate sequence have exactly $L(w)$ entries, one proof for each $\mathrm{hd}(\mathrm{tl}^i(w))$, $i < L(w)$. If pushing a onto w has $a = w = 0$, retain the empty sequence. Otherwise $L(p(a,w)) = L(w) + 1$ and prepend the proof of a. A positive pop removes the first certificate. A zero pop returns a fixed proof of the truth translation and keeps the empty sequence. All operations are primitive recursive, with their contracts provable by PA induction. □

This is a refinement of the existing reverse-proof plan. It avoids an unrecorded stack-length witness and handles the exceptional pairing value explicitly. The actual Shift halt condition is $t + h = 1$, so a certificate invariant for both t and h yields a proof of whichever component equals one.

7.2 Proof-producing arithmetic interpretation

Interpret membership by the binary-bit predicate

$$B(s,z) \;\leftrightarrow\; \exists u, v\,[s = 2^{z+1}u + 2^z + v \;\wedge\; v < 2^z].$$

Powers of two and other primitive-recursive coding operations abbreviate their definitions in a conservative extension of PA. PA proves $B(s,z) \to z < s$, and hence $\neg B(s,s)$. Equality is arithmetic equality, and the transposed universal constructor is decoded directly. Code zero is a fixed tautology τ; the alias $I(0,0) = 0$ is handled by a fixed proof of $\tau \leftrightarrow (\tau \to \tau)$.

For an input arithmetic formula φ, full induction supplies a proof template for bounded bit comprehension:

$$\forall m\, \exists s < 2^{m+1}\, \forall z \leq m\, [B(s, z) \leftrightarrow \varphi(z)].$$

The constructor inserts φ into a fixed induction argument; it does not decide its truth. This gives the finite-set witness for the pointed-separation component of the guarded gate. The two vacuity clauses are valid when the guard excludes the relevant variable labels. The companion note spells out the distinct-label and equal-label cases and attributes the direct bit interpretation to the existing bridge work. The new contribution here is the explicit canonical-stack repair, not a claim to have completed the entire internal proof compiler.

7.3 Reverse trace extraction is a real obligation

Suppose, inside PA, every nonhalt source instruction started at a valid frame has a positive exact physical simulation segment ending at its valid successor frame, with no halt at any point of that segment. Suppose coded source executions of arbitrary internal finite length are available, with the source halt status totalized, and exact startup and physical trace determinism are available as well. A literal halting trace of remaining length t cannot coexist with $t + 1$ consecutive nonhalt source steps: composing their positive segments gives a physical nonhalting segment longer than the alleged halting time, and determinism identifies its prefix with the alleged trace. This supplies a bounded source-halt witness.

The hypotheses matter. A forward interface allowing either a successor frame *or an intervening physical halt* cannot provide this conclusion. The coded segments, exact state projection, scheduler, partially completed macros, and proof-code constructors still need their own PA correctness proofs. Neither source graph equality nor the local search receipts fill that gap automatically.

8 Reproduction and artifact map

`data/`
: Pinned PA196 table and source, with source identities and license notices.

`comparison/`
: All nine designated U208 comparison tables, exact canonical encodings and maps, the two U208 source layouts, and replay scripts.

`source_reductions/`
: The 157-node, layout-estimate 158-node, and minimal 158-node witnesses; independent full-graph audit; deterministic reconstruction of the minimal factorization; and search receipts. The supplied Lean source data are explicitly marked as not compiled in this session.

`search/`
: Finite register abstractions, source-conditioned tape contexts, bounded fold synthesis, controls, and receipts. More expensive screens are optional reproductions; the saved exact witnesses do not require rerunning an optimizer.

`logic/`
: Native HA reference checker, demonstration certificate, nine implementation checks, mathematical specification, and reverse-proof obligations.

`verify.py`
: Fast packet checks: input identities, complete graph equality, canonical comparison, source census, and reference-checker controls.

Run `python3verify.py` from the extracted packet directory. The text `README.txt` gives the optional larger search commands and required tools. Build this report by running

```
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

twice. The archive includes the compiled PDF. No generated test receipt is being substituted for a Lean/Rocq proof object.

9 Further research questions

1. Can the 157-node source tree be allocated with at most 146 source-reader states while preserving the engine and routing budgets, or can a larger source tree admit better contextual sharing? The objective should count all decoder-helper roles, including nodes no longer visited as source readers.
2. Can the (D, C) factorization be combined with an alternative helper for native state 69 outside the tested eleven-atom interval? A promising allocator would jointly choose source association, helper behavior, and state sharing, instead of fixing all old compatible assignments first.
3. Can a semantic register invariant beyond the tested finite products and pairwise sign abstractions remove a complete repeated source block, so that the reduction survives grammar compression and physical allocation?
4. Can the return scan be shortened together with a source/path encoding that rules out the transient zero patterns produced by decrement-zero updates? A bound on final source paths alone is insufficient.
5. What binary state budget results from compiling the native HA checker, including enumeration and substitution? This is the missing quantitative experiment needed before the ConciseIPRA transfer becomes a competitive machine construction.
6. Can the canonical proof-stack invariant be integrated with an exact reverse trace extractor and a proof-producing PA interpretation, completing the internal biconditional for the existing PA196 before transferring it to any smaller descendant?

The established new numerical improvement in this packet is **159 to 157 distinct internal source nodes**. The requested **196 to at most 195 working-state improvement remains open in this work**.

References

[1] James Chang, *Nonhalting and PA-independence for a 196-state Turing machine*, pinned source revision `407cdc356f6b586635e71ac92be394255f4bfea4`. `https://github.com/jamesyc/BusyBeaver/blob/407cdc356f6b586635e71ac92be394255f4bfea4/paper/pa196.tex`.

[2] James Chang, formal theorem inventory, same revision. `https://github.com/jamesyc/BusyBeaver/blob/407cdc356f6b586635e71ac92be394255f4bfea4/docs/result.md`.

[3] James Chang, `PA196/Independence.lean`, same revision. `https://github.com/jamesyc/BusyBeaver/blob/407cdc356f6b586635e71ac92be394255f4bfea4/PA196/Independence.lean`.

[4] James Chang, requested PA209 snapshot. `https://github.com/jamesyc/BusyBeaver/tree/69d98241a721ead20b4f0b17df9062950b40fbfe`.

[5] ProveIt, incoming U208 submissions, pinned revision `f1f076e31995f10460838d1846456f4bb9c8ae80`. `https://github.com/VladimirReshetnikov/ProveIt/tree/f1f076e31995f10460838d1846456f4bb9c8ae80/docs/incoming`.

[6] ProveIt, existing PA196/U208 comparison, same revision. `https://github.com/VladimirReshetnikov/ProveIt/blob/f1f076e31995f10460838d1846456f4bb9c8ae80/Computability/BusyBeaver/Research/PA/pa196-vs-u208/COMPARISON.md`.

[7] ProveIt, existing internal-PA bridge audit, same revision. `https://github.com/VladimirReshetnikov/ProveIt/blob/f1f076e31995f10460838d1846456f4bb9c8ae80/Computability/BusyBeaver/Research/PA/pa196-vs-u208/PA-BRIDGE.md`.

[8] ProveIt, *ConciseIPRA*, especially the witness-rule report, readable checker, and native Lean developments, same revision. `https://github.com/VladimirReshetnikov/ProveIt/tree/f1f076e31995f10460838d1846456f4bb9c8ae80/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`.